3522 BULUT BİLİŞİM DERSİ

PROJE 1

RAPORU

Çift Katmanlı Web Uygulaması

(Web API + Frontend)

Kullanılan Teknolojiler	Proje Özeti
• Node.js + Express.js (Backend API) • React.js + Vite (Frontend SPA) • SQLite / better-sqlite3 (Veritabanı) • AWS EC2 + Nginx (Bulut Dağıtım)	Glassmorphism arayüzlü, RESTful API destekli çift katmanlı yapılacaklar listesi uygulaması. AWS EC2 üzerinde Nginx reverse proxy ile dağıtıma alınmıştır.

GitHub Proje Reposu

github.com/SALIM-20291313/bulut-bilisim-proje-1

Hazırlayan	Ders Sorumlusu	Öğretim Yılı / Tarih
SALİM CAN ŞEN 20291313	**ENVER BAĞCI**	**2025 – 2026** Nisan 2026

İÇİNDEKİLER

1.	**Proje Özeti ve Temel Amaçlar**	**3**
2.	**Sistem Mimarisi ve Kullanılan Teknolojiler**	**3**
2.1	Backend Katmanı	3
2.2	Frontend Katmanı	4
2.3	Mimarinin Şematik Gösterimi	4
3.	**Veritabanı Modeli ve Geliştirme Süreci**	**5**
3.1	SQL Tablo Şeması	5
3.2	Geliştirme Adımları	5
4.	**Karşılaşılan Mimari Sorunlar ve Çözümler**	**6**
5.	**Bulut Dağıtım Stratejisi ve AWS Entegrasyonu**	**7**
5.1	AWS Servis Yapılandırması	7
5.2	Port Mimarisi ve Güvenlik Grupları	8
5.3	Nginx Reverse Proxy Yapılandırması	8
6.	**Sonuç ve Değerlendirme**	**9**

1. Proje Özeti ve Temel Amaçlar

Günümüz dijital yaşantısında zaman ve iş yönetimi en temel ihtiyaçlardan biridir. Bu projenin çekirdek fikri, yalnızca standart bir yapılacaklar listesi üretmek değil; bu fikri endüstri standartlarına uygun bir iki katmanlı (Two-Tier) bulut bilişim mimarisi üzerinden modellemektir.

Proje; sunum katmanı (Frontend) ile veri/iş mantığı katmanının (Backend) birbirinden tamamen ayrıştırıldığı, bulut üzerinde dağıtıma hazır bir SPA (Single Page Application) mimarisi olarak tasarlanmıştır.

Projenin Temel Hedefleri

İzole İstemci-Sunucu Yapısı: Frontend ve Backend ayrı portlarda, Stateless API entegrasyonuyla çalışır • Kapsamlı Görev Yönetimi: Öncelik (Priority), son teslim tarihi (Deadline), tamamlanma oranı izleme • Modern UI/UX: Glassmorphism tasarım dili, SPA deneyimi, sayfa yenilenmeden arama/sıralama • Bulut Dağıtımı: AWS EC2 üzerinde Nginx reverse proxy ile production ortamı

2. Sistem Mimarisi ve Kullanılan Teknolojiler

Proje; bulut ortamında dağıtıma hazır, birbirinden bağımsız çalışan iki ana katmandan oluşmaktadır. Her katman kendine ait port üzerinde çalışır ve birbirleriyle yalnızca HTTP/JSON aracılığıyla iletişim kurar.

2.1 Backend Katmanı (Katman 1)

- **Çalışma Ortamı:** Node.js v18+
- **Web Framework:** Express.js (^4.18) — RESTful API yönlendirmeleri
- **CORS Yönetimi:** cors middleware — çapraz port isteklerini karşılar
- **Ortam Değişkenleri:** dotenv — .env dosyasından konfigürasyon yükler
- **Veritabanı Sürücüsü:** better-sqlite3 — senkron, hızlı SQLite bağlantısı
- **Çalışma Portu:** 5000

2.2 Frontend Katmanı (Katman 2)

- **Framework:** React.js ^18.x — bileşen tabanlı SPA mimarisi
- **Build Aracı:** Vite — hızlı geliştirme ortamı ve üretim derleme
- **HTTP İstemcisi:** Axios — Backend REST API ile iletişim
- **State Yönetimi:** React Hooks (useState, useEffect) — yerel state
- **Tasarım Dili:** Glassmorphism — şeffaflık, bulanıklık, derinlik efektleri
- **Çalışma Portu:** 3000

2.3 Mimarinin Şematik Gösterimi

Aşağıdaki tablo sistemin port bazlı izolasyonunu ve katmanlar arasındaki veri akışını göstermektedir:

FRONTEND (React.js)	Port 3000 | Kullanıcı Arayüzü (SPA) · Axios HTTP İstekleri · State Management
↓ HTTP / REST API	JSON formatında istek ve yanıt
BACKEND (Node.js/Express)	Port 5000 | RESTful API Endpoints · İş Mantığı (Controller) · CORS, dotenv, Middleware
↓ SQL Sorguları	better-sqlite3 kütüphanesi ile senkron sorgular
VERİTABANI (SQLite)	todos tablosu | Yerel .db dosyası | CRUD operasyonları

3. Veritabanı Modeli ve Geliştirme Süreci

Geliştirme süreci veritabanı modellemesiyle başlamış, API entegrasyonu ve ardından UI inşasıyla devam etmiştir. Sistemdeki tüm görevler, gelişmiş parametreler barındıran tekil bir todos tablosunda tutulmaktadır. SQLite tercih edilmesinin temel nedeni hafifliği ve sunucu gerektirmeden çalışabilmesidir.

3.1 SQL Tablo Şeması

```
CREATE TABLE IF NOT EXISTS todos (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  title        TEXT    NOT NULL,
  description  TEXT    DEFAULT '',
  completed    INTEGER DEFAULT 0,    -- 0: bekliyor, 1: tamamlandı
  priority     TEXT    DEFAULT 'medium',  -- low | medium | high
  due_date     TEXT,                 -- Son teslim tarihi
  created_at   TEXT    DEFAULT (datetime('now', 'localtime')),
  updated_at   TEXT    DEFAULT (datetime('now', 'localtime'))
);
```

3.2 Geliştirme Adımları

Geliştirme süreci katmanlı bir yaklaşımla yürütülmüştür:

- **Model Katmanı:** SQLite veritabanı kuruldu ve CRUD operasyonları için senkron sorgular yazıldı (better-sqlite3 ile).
- **Controller ve Route Katmanı:** HTTP isteklerini karşılayacak iş mantıkları (Controller) ve endpoint'ler (Route) express.Router() ile modüler hale getirildi.
- **Arayüz Entegrasyonu:** React State yapısı kurularak useEffect üzerinden veriler anlık olarak ekrana basıldı. Axios ile CRUD işlemleri bağlandı.
- **UI/UX Geliştirme:** Glassmorphism bileşenleri, arama/filtreleme/sıralama özellikleri ve görev öncelik renk sistemi eklendi.

4. Karşılaşılan Mimari Sorunlar ve Çözümler

Bu ölçekte dağıtık bir sistem geliştirilirken karşılaşılan temel mühendislik problemleri ve uygulanan çözümler aşağıda özetlenmiştir:

Problem	Sorun	Çözüm
CORS & Port Çarpışması	React (Port 3000) ve Node.js (Port 5000) farklı portlarda çalışınca tarayıcı Preflight isteklerini engelledi.	app.use(cors({ origin: '*' })) Middleware katmanı eklenerek iki izole sunucunun güvenli iletişimi sağlandı.
DB Migration Riski	due_date özelliği eklendiğinde aktif verilerle dolu SQLite tablosunu CREATE TABLE ile ezmek veri kaybına yol açacaktı.	try-catch bloğu ile güvenli ALTER TABLE sorgusu (ALTER TABLE todos ADD COLUMN due_date TEXT) çalıştırıldı.
State & Performans	Arama, filtreleme ve sıralama için sürekli Backend'e istek atmak gecikmelere (lag) sebep oldu.	Veriler yalnızca açılışta çekildi. Tüm filtreleme .filter() ve .sort() ile Frontend'de milisaniyeler içinde yapıldı.

CORS Çözümü — Kod

```
// server.js — CORS Middleware
const cors = require('cors');
app.use(cors({ origin: '*' }));  // Tüm kaynaklardan erişime izin ver
```

Database Migration Çözümü — Kod

```
// db.js — Güvenli sütun ekleme (veri kaybı olmadan)
try {
  db.exec(`ALTER TABLE todos ADD COLUMN due_date TEXT`);
} catch (err) {
  // Sütun zaten varsa hata yoksay (migration güvenli)
}
```

5. Bulut Dağıtım Stratejisi ve AWS Entegrasyonu

Projenin yerel ortamdan buluta taşınma sürecinde yüksek erişilebilirlik sağlamak amacıyla AWS EC2 (Elastic Compute Cloud) sanal sunucu altyapısı tercih edilmiştir. Sistem, iki katmanlı mimarisine uygun olarak bulut üzerinde de tamamen izole portlar üzerinden haberleşmektedir.

5.1 AWS Servis Yapılandırması

Servis	Bileşen	Açıklama
AWS EC2	t2.micro / t3.small	Ubuntu 22.04 LTS sanal sunucu, iki uygulama aynı instance üzerinde
Security Groups	Inbound Rules	HTTP(80), HTTPS(443), Port 3000, Port 5000 izinleri açıldı
Nginx	Reverse Proxy	Dış trafik Nginx ile karşılanır, ilgili portlara yönlendirilir

PM2	Process Manager	Node.js uygulamaları arka planda sürekli çalışır, crash recovery
Elastic IP	Statik IP	Sunucu yeniden başlatıldığında IP değişmez, sabit erişim

5.2 Port Mimarisi ve Güvenlik Grupları

AWS Console üzerinden yapılandırılan Inbound Rules (Gelen Trafik Kuralları) ile ağ güvenliği sağlanmıştır:

- **Port 3000:** React.js arayüzünün dış dünyaya servis edildiği, istemci bağlantılarının karşılandığı ana port
- **Port 5000:** Node.js/Express REST API sunucusunun dinlediği ve veritabanı işlemlerini yürüttüğü izole port
- **Port 80/443:** HTTP ve HTTPS standart erişim portları — Nginx tarafından karşılanır
- **SSH (22):** Güvenli uzak sunucu erişimi — yalnızca yetkili IP adreslerine açık

5.3 Nginx Reverse Proxy Yapılandırması

AWS EC2 sunucusuna gelen dış trafik Nginx ile karşılanmış, gelen istekler arka plandaki ilgili portlara güvenli biçimde yönlendirilmiştir. Bu yapı port karmaşasını önlerken aynı zamanda SSL termination noktası olarak da görev yapar.

```
# /etc/nginx/sites-available/proje1
server {
    listen 80;
    server_name <EC2-PUBLIC-IP>;

    # Frontend (React) — Port 3000
    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
    }

    # Backend (Express API) — Port 5000
    location /api/ {
        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
    }
}
```

Gelecek Ölçekleme Planı

Bu dağıtım mimarisi; Docker konteynerleri ile servis izolasyonu, AWS RDS (PostgreSQL) ile tam teşekküllü yönetilen veritabanı, ve AWS Load Balancer ile yüksek erişilebilirlik sağlanacak şekilde kolayca genişletilebilir.

6. Sonuç ve Değerlendirme

Bu proje kapsamında endüstri standartlarına uygun, iki katmanlı bir bulut web uygulaması başarıyla geliştirilmiş ve AWS EC2 üzerinde production ortamına alınmıştır. Frontend ve Backend'in port bazlı izolasyonu, Stateless API tasarımı ve Nginx reverse proxy mimarisi modern bulut uygulamalarının temel prensiplerini somut biçimde uygulamaya koymuştur.

Proje; CORS yönetimi, veritabanı migration, Frontend state optimizasyonu ve bulut ağ güvenliği gibi gerçek dünya mühendislik problemleriyle yüzleşme ve çözüm üretme fırsatı sunmuştur. Glassmorphism tasarım diliyle inşa edilen modern SPA arayüzü, kullanıcı deneyimini ön plana çıkarmaktadır.

Sistem; Docker konteynerleri, AWS RDS ve Load Balancer gibi tam teşekküllü bulut servisleriyle kolayca ölçeklenebilecek sağlam bir mimari temele sahiptir.

GitHub Proje Reposu

Proje kodları, rapor ve çalışma logları aşağıdaki Git reposunda sürüm kontrolü ile yönetilmiştir.

https://github.com/SALIM-20291313/bulut-bilisim-proje-1